

Introduction To Bison

Solved:

Solved.y

```
%{  
    #include<stdio.h>  
    #include<stdlib.h>  
  
    int yylex();  
    int yyerror(char *msg);  
    extern FILE *yyin;  
}%  
  
%token NUMBER ID NL  
  
%left '+'  
%left '*'  
  
%%  
stmt : exp NL { printf("Valid Expression\n"); exit(0); }  
    ;  
  
exp : exp '+' term  
    | term  
    ;  
  
term : term '*' factor  
    | factor
```

```
;
```

```
factor : ID
```

```
      | NUMBER
```

```
;
```

```
%%
```

```
int yyerror(char *msg)
```

```
{
```

```
    printf("Invalid Expression\n");
```

```
    exit(0);
```

```
}
```

```
int main ()
```

```
{
```

```
    printf("Enter the expression\n");
```

```
    yyparse();
```

```
    return 0;
```

```
}
```

Solved.I

```
%{
```

```
    #include "solved.tab.h" //filename.tab.h : here both flex and bison
```

```
%}
```

```
%%
```

```
[0-9]+ {return NUMBER; }
```

```
\n {return NL ;}
```

```
[a-zA-Z][a-zA-Z0-9_]* {return ID; }
. {return yytext[0]; }
%%
```

```
int yywrap(){
    return 1;
}
```

Screenshot:

```
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ bison -d solved.y
solved.y:29 parser name defined to default : "parse"
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ flex solved.l
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ gcc lex.yy.c solved.tab.c -o output
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the expression
a+b
Valid Expression
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the expression
a*b
Valid Expression
```

Q1:

Q1.y

```
%{
    #include <stdio.h>
    #include <stdlib.h>

    int yylex();
    void yyerror(char *s);
}%
```

```
%token TYPE ID SC COMMA NL
```

%%

```
start: decl NL { printf("Valid Declaration Statement\n"); exit(0); }
```

```
;
```

```
decl : TYPE list SC
```

```
;
```

```
list : ID
```

```
    | list COMMA ID
```

```
;
```

%%

```
void yyerror(char *s) {
```

```
    printf("Invalid Declaration Statement\n");
```

```
    exit(0);
```

```
}
```

```
int main() {
```

```
    printf("Enter a declaration statement :\n");
```

```
    yyparse();
```

```
    return 0;
```

```
}
```

Q1.1

```
%{
```

```
    #include "q1.tab.h"
```

```
%}
```

```

%%
"int"|"float"|"char"|"double" { return TYPE; }
[a-zA-Z][a-zA-Z0-9]* { return ID; }
"," { return COMMA; }
";" { return SC; }
"\n" { return NL; }
[ \t] { ; }
. { return yytext[0]; }
%%

```

```

int yywrap() {
    return 1;
}

```

Screenshot:

```

6CSEC1@db1-23:~/Documents/CD_230905152/L10$ bison -d q1.y
q1.y:21 parser name defined to default : "parse"
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ flex q1.l
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ gcc lex.yy.c q1.tab.c -o output
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter a declaration statement :
int a,b;
Valid Declaration Statement
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter a declaration statement :
int a
Invalid Declaration Statement
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter a declaration statement :
char a,b;
Valid Declaration Statement
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ █

```

Q2:

Q2.y

```

%{ #include <stdio.h> #include <stdlib.h>

int yylex();
void yyerror(char *s);

%}

%token IF ELSE ID NUM RELOP ASSIGN

%% start : IF_STMT { printf("Valid Decision Making Statement\n"); exit(0); } ;

IF_STMT : IF '(' CONDITION ')' BLOCK | IF '(' CONDITION ')' BLOCK ELSE BLOCK ;

BLOCK : STMT ';' | '{' STMT_LIST '}';

STMT_LIST : STMT_LIST STMT ';' | STMT ';' | /* empty */ ;

CONDITION : EXPR RELOP EXPR | EXPR ;

STMT : ID ASSIGN EXPR | IF_STMT ;

EXPR : ID | NUM ;

%%

void yyerror(char *s) { printf("Invalid Decision Making Statement\n"); exit(0); }

int main() { printf("Enter the decision statement:\n"); yyparse(); return 0; }

```

Q2.I

```

%{
#include "q2.tab.h"
#include <string.h>

%}

%%

"if"          { return IF; }

```

```

"else"          { return ELSE; }
[0-9]+          { return NUM; }
[a-zA-Z][a-zA-Z0-9]* { return ID; }
"<="|">="|"=="|"!="|"<"|>" { return RELOP; }
"+="|"-=|"*="|"/="|"=" { return ASSIGN; }
[ \t\n]         { ; }
.               { return yytext[0]; }

%%

```

```

int yywrap() {
    return 1;
}

```

Screenshot:

```

6CSEC1@db1-23:~/Documents/CD_230905152/L10$ flex q2.1
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ gcc lex.yy.c q2.tab.c -o output
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the decision statement:
if(a>5) a+=5;
else a-=5;
Valid Decision Making Statement
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the decision statement:
int a=5;
Invalid Decision Making Statement

```

Q3/Solved

Q3.y

```

%{

#include<stdio.h>

#include<stdlib.h>


int yylex();

```

```

    int yyerror(char *msg);
    extern FILE *yyin;
%}

%token NUMBER ID NL
%left '+'
%left '-'
%left '*'
%left '/'

%%

stmt : exp NL { printf("Valid Expression\n"); exit(0); }
    ;

exp : exp '+' term
    | exp '-' term
    | term
    ;

term : term '*' factor
    | term '/' factor
    | factor
    ;

factor : ID
    | NUMBER
    ;

```


%%

```
int yyerror(char *msg)
{
    printf("Invalid Expression\n");
    exit(0);
}
```

```
int main ()
{
    printf("Enter the expression\n");
    yyparse();
    return 0;
}
```

Q3.I

```
%{
    #include<stdio.h>
    #include<stdlib.h>

    int yylex();
    int yyerror(char *msg);
    extern FILE *yyin;
}%
```

%token NUMBER ID NL

%left '+'

%left '-'

%left '*'

%left '/'

%%

stmt : exp NL { printf("Valid Expression\n"); exit(0); }

;

exp : exp '+' term

| exp '-' term

| term

;

term : term '*' factor

| term '/' factor

| factor

;

factor : ID

| NUMBER

;

%%

int yyerror(char *msg)

{

printf("Invalid Expression\n");

exit(0);

}

```

int main ()
{
    printf("Enter the expression\n");
    yyparse();
    return 0;
}

```

Screenshot:

```

6CSEC1@db1-23:~/Documents/CD_230905152/L10$ bison -d q3.y
q3.y:33 parser name defined to default : "parse"
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ flex q3.l
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ gcc lex.yy.c q3.tab.c -o output
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the expression
a+b-c*d/e
Valid Expression
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the expression
a$b
Invalid Expression
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ ./output
Enter the expression
ab-cd
Valid Expression
6CSEC1@db1-23:~/Documents/CD_230905152/L10$ █

```

Q4

Q4.y

```
%{ #include <stdio.h> #include <stdlib.h>
```

```

int yylex();
int yyerror(char *msg);

```

```
%}
```

```
%token NUMBER ID NL
```

```

%% input : /* empty */ | input line ;

line : NL | exp NL { printf("Valid Postfix Expression\n"); } ;

exp : NUMBER | ID | exp exp '+' | exp exp '-' | exp exp '*' | exp exp '/' | exp exp '^' ;

%%

int yyerror(char *msg) { printf("Invalid Expression\n"); return 0; }

int main() { printf("Enter postfix expression:\n"); yyparse(); return 0; }

```

Q4.I

```

%{
    #include "q4.tab.h"
}%

%%

[0-9]+      { return NUMBER; }
[a-zA-Z]    { return ID; }
\n          { return NL; }
[ \t]       { ; }
"+"|"-"|"*"|"/"|"^" { return yytext[0]; }
.           { return yytext[0]; }

%%

int yywrap() {
    return 1;
}

```

Screenshot:

```
6CSEC1@dbl-23:~/Documents/CD_230905152/L10$ bison -d q4.y
q4.y:29 parser name defined to default : "parse"
6CSEC1@dbl-23:~/Documents/CD_230905152/L10$ flex q4.l
6CSEC1@dbl-23:~/Documents/CD_230905152/L10$ gcc lex.yy.c q4.tab.c -o output
6CSEC1@dbl-23:~/Documents/CD_230905152/L10$ ./output
Enter postfix expression:
ab+
Valid Postfix Expression
a-b
Invalid Expression
```